

Montar software con Claude Code

Instrucciones de montaje para pasar de una idea a una aplicación que funciona en tu portátil. Sin saber programar.

PIEZAS INCLUIDAS

- 1× Claude Code, el agente
- 2× `AGENTS.md`, el contexto fijo
- 3× skills del repositorio
- 6× comandos `/opsx` de OpenSpec
- 1× enunciado: app de notas

Lo que vamos a ver, en el orden en que lo vas a usar

1

Claude Code

Qué es, dónde se ejecuta, qué puede tocar y cómo recibe contexto con `AGENTS.md`.

2

Skills y OpenSpec

Manuales que se cargan cuando hacen falta, y los comandos `/opsx` para trabajar con especificaciones.

3

Describir bien

Cómo explicar una funcionalidad para que la propuesta salga bien a la primera.

4

El ciclo completo

De la idea a la aplicación funcionando y probada en tu máquina.

Un chat responde. Un agente, además, hace cosas.

Claude Code es un **agente**: una IA que, además de contestar, puede trabajar sobre tu proyecto.

- **Lee** los archivos de la carpeta que abres.
- **Escribe y edita** código y documentos.
- **Ejecuta comandos** en la terminal: Git, Docker, instalar dependencias...
- **Comprueba** lo que ha hecho y corrige si algo falla.

CHAT NORMAL

"¿Cómo creo una tabla en una base de datos?"

Te da una explicación. Lo aplicas tú.

CLAUDE CODE

"Añade una tabla de notas a la base de datos."

Mira tu proyecto, escribe el cambio, lo ejecuta y te enseña qué ha tocado.

Trabaja en tu portátil, dentro de la carpeta que eliges

TU PORTÁTIL

App de escritorio → pestaña **Code** → **Local** → eliges una carpeta.

- Leer** Los archivos de esa carpeta y sus subcarpetas.
- Editar** Según el modo de permisos que elijas.
- Comandos** Se ejecutan en tu terminal. Te pide permiso antes.
- Fuera** Otras carpetas del ordenador: solo si le das permiso.

Manual pregunta todo · **Accept edits** aplica cambios y tú revisas · **Plan** solo propone



lo que escribes y lo que lee



respuestas y acciones

EL MODELO (EN LA NUBE)

La "inteligencia" está en los servidores de Anthropic. Ahí se envía lo que escribes y lo que Claude lee de tus archivos.

Por eso: nada de contraseñas ni datos de clientes en los archivos del proyecto. Van en `.env`, que no se sube.

AGENTS.md : lo que siempre debe saber

Las reglas del proyecto se escriben una vez, en `AGENTS.md`. Es un nombre estándar que entienden varias herramientas de IA.

- Claude Code no busca `AGENTS.md`: lee `CLAUDE.md`. Por eso cada `CLAUDE.md` solo contiene `@AGENTS.md`, que le dice "carga ese archivo".
- **En cascada:** el de la raíz se carga al abrir. El del taller, en cuanto Claude trabaja con archivos de esa carpeta. Si chocan, manda el del taller.
- **Abre siempre la raíz del repositorio.** Si abres la carpeta del taller, se pierden las reglas generales.

<code>AGENTS.md</code>	← reglas de todo el repo
<code>CLAUDE.md</code>	← solo: <code>@AGENTS.md</code>
<code>.claude/skills/</code>	← skills (siguiente bloque)
<code>workshops/</code>	
└─ <code>01-ia-contexto-openspec/</code>	
└─ <code>AGENTS.md</code>	← reglas del ejercicio
└─ <code>CLAUDE.md</code>	← solo: <code>@AGENTS.md</code>
└─ <code>brief/</code>	← qué construir
└─ <code>proyecto/</code>	← tu código

QUÉ VA EN UN AGENTS.MD

Convenciones, restricciones, dónde está cada cosa, qué no hacer. **No** va: documentación larga que solo hace falta a veces, ni secretos.

Una skill es un manual que se carga solo cuando hace falta

Cada skill empieza con una descripción de **cuándo usarla**. Claude lee esas descripciones y decide cuál aplica según lo que le pidas.

¿POR QUÉ NO METERLO TODO EN AGENTS.MD?

Porque el contexto es limitado. `AGENTS.md` es lo que **siempre** aplica; una skill, lo que aplica **en una situación concreta**. Así la IA no carga instrucciones que no necesita.

LAS DE ESTE REPOSITORIO

git-basico

Ramas, commits, deshacer, conflictos, Pull Request. Prueba: *"me he liado con git, ayúdame"*.

entorno-local

Docker, puertos ocupados, base de datos, el archivo `.env`.

revision-de-entrega

Comprueba tu proyecto contra las restricciones. Prueba: *"revisa mi entrega"*.

OpenSpec instala seis comandos, cada uno con su skill

Se escriben en el chat de Claude Code empezando por `/.` `openspec init` se lanza una vez, en la raíz del repositorio.

`/opsx:explore`

`openspec-explore`

Pensar en voz alta: opciones, preguntas, pros y contras. **No escribe nada.**

`/opsx:propose`

`openspec-propose`

Escribe el plan: propuesta, especificación, diseño y lista de tareas. **No toca código.**

`/opsx:apply`

`openspec-apply-change`

Implementa las tareas del plan, una a una. Aquí se escribe el código.

`/opsx:archive`

`openspec-archive-change`

Cierra el cambio y actualiza las especificaciones para que describan cómo es el sistema hoy.

`/opsx:update`

`openspec-update-change`

Corrige un plan ya escrito y mantiene coherentes sus documentos. **No toca código.**

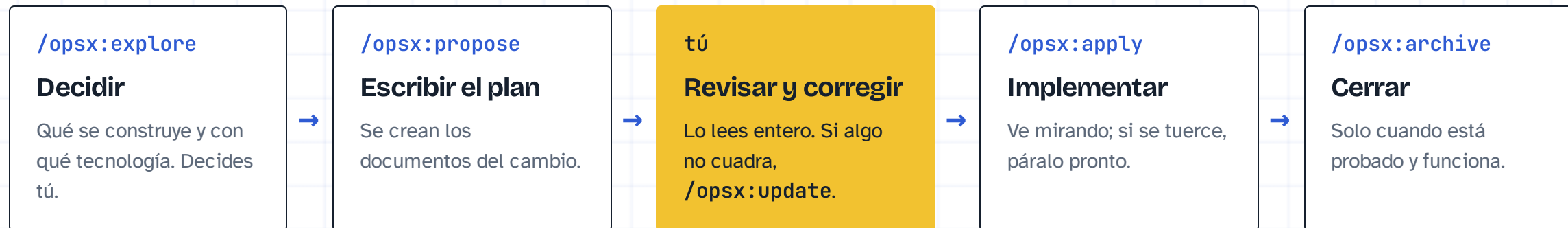
`/opsx:sync`

`openspec-sync-specs`

Pasa la especificación del cambio a las especificaciones principales sin archivar todavía.

Borde oscuro: el ciclo principal · borde claro: de apoyo, cuando hacen falta.

Primero se acuerda qué hacer, por escrito. Después se programa.



LO QUE GENERA PROPOSE

proposals.md specs/.../spec.md design.md tasks.md

Qué y por qué · qué debe hacer el sistema · cómo se construye · pasos para hacerlo.

LA IDEA CLAVE

Cambiar un párrafo de la propuesta cuesta treinta segundos. Cambiar una decisión ya programada, una tarde.

La propuesta es tan buena como la descripción que le das

POCO ÚTIL

"Hazme una app de notas."

La IA tiene que inventarse el resto: qué campos, qué pasa si falla algo, qué queda fuera. Lo que se invente, lo vas a tener que corregir.

ÚTIL

"Una app web para que **una sola persona** cree, vea, edite y borre notas. Cada nota tiene **título obligatorio de máximo 120 caracteres** y contenido opcional. Si el título está vacío, **no se guarda y se explica por qué**. Antes de borrar, **pide confirmación**. **Fuera de alcance**: login, etiquetas y búsqueda. **Está terminada cuando** las notas siguen ahí después de apagar y encender."

Para quién y para qué

Quién la usa y qué problema le resuelve.

Qué entra y qué no

Una lista de lo que queda fuera evita que crezca sola.

Cuándo está terminada

Pruebas concretas que puedas hacer tú en el navegador.

Buenas prácticas para escribir y revisar el plan

- **Describe comportamiento, no tecnología.** "Las notas sobreviven a un reinicio", no "usa PostgreSQL". La tecnología se decide en `explore` y va en `design.md`.
- **Cuenta los casos difíciles.** Título vacío, lista vacía, una nota que ya no existe. Son los que separan algo terminado de algo a medias.
- **Usa ejemplos concretos** con valores reales: "121 caracteres", no "un título largo".
- **Un cambio cada vez.** Si te sale una lista enorme, pártela en varios cambios.
- **Si no lo sabrías explicar, pregunta.** "¿Qué es esto y por qué lo has elegido?"
- **Corrige el plan, no el código.** Si cambias de idea, `/opsx:update` antes de seguir implementando.

ASÍ QUEDA UN CASO EN `SPEC.MD`

```
### Requirement: Título obligatorio
El sistema SHALL impedir guardar una nota
sin título o con un título de solo espacios.

#### Scenario: Título vacío
- WHEN la persona guarda una nota con
  el título "  "
- THEN la nota no se guarda
- AND se muestra "El título es obligatorio"
```

Cada caso **CUANDO → ENTONCES** es algo que puedes comprobar tú en el navegador. Si no se puede comprobar, está mal escrito.

Instrucciones de montaje: de la idea a la aplicación probada

PENSAR

1 La idea
Escríbela: para quién, qué hace, qué no.

2 Tu rama
`git checkout -b taller1/<nombre>`

PLANIFICAR

3 Explorar
`/opx:explore`

4 Proponer y revisar
`/opx:propose` y lo lees entero.

CONSTRUIR

5 Implementar
`/opx:apply`

6 Guardar avances
Un commit cada vez que algo funciona.

PROBAR

7 Levantar en local
`docker compose up`

8 Probar de verdad
La lista de "terminada", en el navegador. Luego "revisa mi entrega".

ENTREGAR

9 Cerrar el cambio
`/opx:archive`

10 Subir y pedir revisión
`git push` y Pull Request.

 Pasos en los que decides o compruebas tú  Pasos que hace Claude contigo

"Que arranque" no es "que funcione"

En **local** significa en tu portátil: la app y su base de datos corren en contenedores de Docker.

```
$ docker compose up      # levantar app y base de datos
$ docker compose down    # apagar los contenedores
$ docker compose up      # ¿siguen tus notas?
```

● `http://localhost:<puerto>`

El **puerto** es el número de puerta por el que se entra a la aplicación. Se configura en tu `.env`.

RECORRE LA LISTA DEL ENUNCIADO

- Se levanta con **un solo comando** en una máquina limpia.
- Sin notas, la pantalla dice que no hay ninguna todavía.
- Creas, abres y editas una nota, y se ve bien.
- Una nota sin título **no se guarda** y te explica por qué.
- Apagas y enciendes: **las notas siguen ahí**.
- Al borrar, pide confirmación.

¿Algo no arranca? Copia el error tal cual y pregunta **por qué pasaba**, no solo cómo arreglarlo. Ahí entra la skill `entorno-local`.

Tres ideas que quedan cuando se apaga el portátil

El contexto manda

El resultado de una IA depende sobre todo de la información que le das.

`AGENTS.md` y las skills son esa información, escrita en archivos.

Acuerda antes de programar

Una propuesta revisada es más barata que código corregido. Lee el plan entero antes de `/opsx:apply`.

Probado, no solo hecho

Algo está terminado cuando has recorrido la lista en el navegador y los datos sobreviven a un reinicio.